

Robust Algorithms for Sketched Least Squares with Iterative Refinement

Luke Triplett, Xinqi Qian, Atharv Sampath

December 15, 2025

Abstract

Randomized sketching has become a standard tool for accelerating large overdetermined least-squares problems, but naive sketch-and-solve methods are limited in accuracy and lack backward stability in finite precision. In this report we study sketched least-squares solvers that combine randomized embeddings with iterative refinement, following recent SPIR and FOSSILS-type frameworks.

We first examine the finite-precision behavior of standard sketch-and-solve algorithms and show numerically that, for nearly consistent systems, their relative residual error exhibits an empirical noise floor around 10^{-13} , well above machine precision. This behavior prevents convergence to the distortion levels predicted by exact-arithmetic sketching theory and reflects unavoidable floating-point error in the sketched system.

We then focus on iterative-refinement schemes that use the sketch only to construct a preconditioner and an initial iterate, while refinement is performed on the full least-squares problem. We compare several sketched preconditioners, including truncated SVD, regularized QR, and Cholesky-based constructions, and study their performance across a range of dimensions, condition numbers, and residual regimes. Our experiments show that iterative refinement substantially improves numerical robustness relative to sketch-and-solve, recovering backward stability even for highly ill-conditioned matrices, and that a hybrid Cholesky/QR strategy provides a favorable tradeoff between robustness and computational cost.

1 Introduction

Overdetermined least-squares problems arise in many applications. Classical direct methods based on QR factorization are reliable but can be expensive for very large m (number of rows). Randomized sketching methods reduce the computational cost by working with a sketched system

$$SA \in \mathbb{R}^{r \times n}, \quad Sb \in \mathbb{R}^r, \quad n \lesssim r \ll m.$$

Only recently have backward stable randomized algorithms been developed [1]. These algorithms use the sketched system to build a preconditioner and an initial iterate for a traditional iterative solver, combined with a few steps of iterative refinement. The resulting meta-algorithm is outlined in Algorithm 1.

Using a sparse sketching matrix, the runtime of the meta-algorithm is $\mathcal{O}(mr + rn^2)$. The $\mathcal{O}(mr)$ term comes from forming the sketch and from applications of the preconditioned operator inside the iterative solver, while the $\mathcal{O}(rn^2)$ term comes from constructing the preconditioner from SA . To guarantee a high-quality sketch, it suffices in theory to choose the sketch dimension $r = \mathcal{O}(n \log n)$.

In the original work, the authors propose using QR decomposition or SVD to compute the preconditioner, and LSQR or the Polyak heavy-ball method as the iterative solver. In this report,

Algorithm 1 General meta-algorithm for fast, backward stable randomized least-squares

Require: Matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$, subroutine ITERATIVESOLVER

Ensure: Approximate least-squares solution $x_2 \in \mathbb{R}^n$

- 1: Draw a sketching matrix S for $[A \ b]$
 - 2: Construct a right preconditioner P from SA (e.g. by forming a factorization of SA or $A^\top S^\top SA$)
 - 3: Compute an initial guess x_0 by (approximately) solving $\min_x \|SAx - Sb\|_2$ using P
 - 4: **for** $i = 0, 1$ **do**
 - 5: $c_i \leftarrow P^\top A^\top (b - Ax_i)$
 - 6: $\delta y_i \leftarrow \text{ITERATIVESOLVER}(z \mapsto P^\top A^\top (A(Pz)), c_i)$
 - 7: $x_{i+1} \leftarrow x_i + P \delta y_i$
 - 8: **end for**
-

we analyze different choices of preconditioner, focusing especially on highly ill-conditioned matrices A , and we also study finite-precision breakdown of sketch-and-solve in nearly consistent systems.

2 Background

We consider the standard least squares problem and denote the exact solution as $x_\star$.

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2, \quad A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m \quad (1)$$

2.1 Sketching

Let $S \in \mathbb{R}^{r \times m}$ be a random sketching matrix. Common choices include Gaussian embeddings and randomized Hadamard transforms. In our experiments, we will use a sparse sign embedding, i.e.

$$S = \zeta^{-1/2} [s_1 \ \cdots \ s_m].$$

The columns $s_j \in \mathbb{R}^r$ have exactly ζ non-zero entries drawn uniformly from $\{1, -1\}$. A sketching matrix is said to have distortion η if

$$(1 - \eta) \|Ax\|_2 \leq \|SAx\|_2 \leq (1 + \eta) \|Ax\|_2, \quad \forall x \in \mathbb{R}^n.$$

Theoretical results show that the previously mentioned methods have distortion $\eta \in (0, 1)$ with high probability so long as $r = O(n \log n)$. The corresponding sketched least-squares problem is

$$\min_{x \in \mathbb{R}^n} \|SAx - Sb\|_2.$$

Solving the sketched problem directly is known as "sketch and solve". This gives an inexpensive but low precision solution to the original least square problem. In particular, theoretical results show that in exact arithmetic

$$(1 - \eta) \|Ax_\star - b\|_2 \leq \|Ax_{sk} - b\|_2 \leq (1 + \eta) \|Ax_\star - b\|_2. \quad (2)$$

We will use the sketch to build a preconditioner rather than to replace the original objective.

2.2 Error Measures

Let $\hat{x}$ be the numerical solution returned by the algorithm. We use the following error metrics to evaluate the solutions returned by our algorithm.

Relative residual error.

$$\text{res}(\hat{x}) := \frac{\|A\hat{x} - b\|_2}{\|Ax_\star - b\|_2} - 1.$$

Forward error.

$$\text{fwd}(\hat{x}) := \frac{\|\hat{x} - x_\star\|_2}{\|x_\star\|_2}.$$

Backward error. For a candidate solution $\hat{x}$, the backward error for the least-squares problem is

$$\text{BE}_\theta(\hat{x}) := \min \left\{ \left\| \begin{bmatrix} \Delta A & \theta \Delta b \end{bmatrix} \right\|_F : \hat{x} = \arg \min_x \|(b + \Delta b) - (A + \Delta A)x\| \right\}.$$

Here, θ controls the penalty of a perturbation to b versus a perturbation to A . Computing $\text{BE}(\hat{x})$ exactly is expensive, so we use the Karlson–Waldén estimate as a proxy. Let $\hat{r} = b - A\hat{x}$. Define

$$\widehat{\text{BE}}_\theta(\hat{x}) := \frac{\theta}{\sqrt{1 + \theta^2 \|\hat{x}\|^2}} \left\| \left(A^\top A + \frac{\theta^2 \|\hat{r}\|^2}{1 + \theta^2 \|\hat{x}\|^2} I \right)^{-1/2} A^\top \hat{r} \right\|.$$

Taking the limit $\theta \rightarrow \infty$ yields the simpler proxy

$$\nu(\hat{x}) := \widehat{\text{BE}}_\infty(\hat{x}) = \frac{1}{\|\hat{x}\|} \left\| \left(A^\top A + \frac{\|\hat{r}\|^2}{\|\hat{x}\|^2} I \right)^{-1/2} A^\top \hat{r} \right\|,$$

and we use $\nu(\hat{x})$ as our backward-error proxy in the experiments below. This limit has the effect of only allowing perturbations to A . It has been shown that this proxy agrees with the true backward error up to a factor of $\sqrt{2}$ [2]. Note that a small backward error does not imply a small forward error in the same way as it does for linear systems. In particular, the condition number of the least squares problem can substantially exceed the ordinary condition number $\kappa_2(A)$. Roughly speaking, one has

$$\text{effective cond. number} = \begin{cases} O(\kappa_2(A)), & \text{if } \|b - Ax_\star\|_2 \ll \|A\|_2 \|x_\star\|_2, \\ O(\kappa_2^2(A)), & \text{if } \|b - Ax_\star\|_2 = O(\|A\|_2 \|x_\star\|_2), \\ \text{unbounded}, & \text{if } \|b - Ax_\star\|_2 \gg \|A\|_2 \|x_\star\|_2. \end{cases}$$

Backward stability is especially important for basic linear algebra subroutines such as least squares, since these routines are called frequently in larger algorithms and their instability can propagate to the overall method.

3 Algorithms

3.1 Sketched preconditioners

First, we may notice that

$$Ax - b = APP^{-1}x - b = APy - b,$$

where $y = P^{-1}x$. So, right preconditioning replaces the original problem

$$\min_x \|Ax - b\|_2$$

with the equivalent problem

$$\min_y \|APy - b\|_2,$$

and we recover $x = Py$ at the end. This motivates preconditioned iterative methods.

Given a sketching matrix $S \in \mathbb{R}^{r \times m}$ we construct the preconditioner from the sketched system SA . The following methods will give mathematically equivalent preconditioners, but may be very different in practice especially when A is extremely ill-conditioned.

3.1.1 Truncated SVD preconditioner

We first compute a thin singular value decomposition

$$SA = U\Sigma V^\top, \quad U \in \mathbb{R}^{r \times n}, \Sigma \in \mathbb{R}^{n \times n}, V \in \mathbb{R}^{n \times n}.$$

In well-conditioned cases we set

$$R = \Sigma V^\top, \quad P = R^{-1}.$$

For very ill-conditioned sketches we truncate small singular values: choose a threshold $\tau > 0$, set

$$\Sigma_\tau = \text{diag}(\sigma_i : \sigma_i \geq \tau),$$

and form

$$R = \Sigma_\tau V^\top, \quad P = R^{-1},$$

which discards nearly singular directions. This is the implementation recommended in [1]. They recommend a threshold of $30u$ which is what we use in our experiments.

3.1.2 QR preconditioner with regularization

In the QR-based construction we compute a reduced QR factorization

$$SA = QR, \quad Q \in \mathbb{R}^{r \times n}, R \in \mathbb{R}^{n \times n},$$

and set $P = R^{-1}$ when SA is reasonably conditioned. For very ill-conditioned sketches we instead form the regularized stacked system

$$\begin{pmatrix} SA \\ \lambda I \end{pmatrix} = \hat{Q} \hat{R}, \quad \lambda > 0,$$

and take

$$P = \hat{R}^{-1}.$$

The Tikhonov term λI damps directions associated with tiny singular values. See B for more details.

3.1.3 Cholesky preconditioner

In the Cholesky-based approach we form the normal matrix

$$G = (SA)^\top SA,$$

compute a Cholesky factorization

$$G = LL^\top,$$

and define

$$R = L^\top, \quad P = R^{-1}.$$

Given any of these choices of P , one can run iterative refinement on the preconditioned system

$$\min_{y \in \mathbb{R}^n} \|APy - b\|_2, \quad \hat{x} = Py.$$

3.2 Iterative refinement

Given a right preconditioner P and an initial guess $x^{(0)}$ (obtained by solving the sketched problem with SA and Sb), we apply a short outer iterative-refinement loop to the full system $\min_x \|Ax - b\|_2$. At refinement step k we form the residual

$$r^{(k)} = b - Ax^{(k)},$$

and update

$$x^{(k+1)} = x^{(k)} + P y^{(k)},$$

where $y^{(k)}$ is computed by running q Polyak or LSQR iterations on a preconditioned normal-equations objective. In our experiments we use two outer refinement steps and return $x^{(2)}$ as the final approximation $\hat{x}$. The iterative refinement is applied to the error of the intermediate solution when inserted into the preconditioned normal equations. Following FOSSILS, our implementation normalizes the columns of A , builds the sketch and preconditioner from the scaled matrix A_{hat} , and then unscales at the end via $\hat{x} = D^{-1}x^{(2)}$ to improve stability.

It is also possible to use conjugate gradient as the inner solver; prior work [1] found the two choices to be comparable in most cases. They also show that this algorithm is backward stable whenever $\kappa_2(A)u \ll 1$, which is supported by our experiments. However, there are potential stability issues for very inconsistent $\|b_{\perp}\| \approx 1$ and very consistent $\|b_{\perp}\| \approx u$ systems which they did not investigate. We will analyze these in more depth in our experiments.

3.3 Our algorithm

It is well known that SVD is the slowest of these factorizations and Cholesky is the fastest. On the other hand, since Cholesky requires forming the sketched normal equations, it can become numerically singular when the condition number of SA is too large, roughly $\kappa_2(SA) \gtrsim u^{-1/2}$. In our implementation we assess this using a simple diagonal-based proxy computed from the Cholesky factor $G = (SA)^{\top}(SA) = LL^{\top}$, namely $\kappa(G) \approx (\max_i L_{ii}/\min_i L_{ii})^2$. In principle, one could instead use a more accurate estimate based on Hager’s 1-norm estimator applied to triangular solves with L^{-1} , which provides a higher-quality proxy for $\|G^{-1}\|_1$. This is what is implemented in LAPACK’s `trcon`. However, in our timing tests the Hager-based estimate took on the same order as the factorization itself (roughly half the Cholesky time), so we did not find it worthwhile in this setting. Because of these tradeoffs, we recommend a hybrid algorithm in which we first attempt to compute the preconditioner using Cholesky, and fall back to QR on the regularized sketched system if the problem appears too ill-conditioned. This algorithm is described in Algorithm 2. We observe that this hybrid strategy greatly improves performance compared to truncated SVD in the regime where the factorization cost dominates ($n \ll m < n^2$).

4 Experiments

In this section we study the numerical behavior of the sketched least-squares algorithms described above. We vary three aspects of the test problems: the dimensions of A , the condition number $\kappa_2(A)$, and the size of the component of b orthogonal to the column space of A . For all experiments we report the relative residual error, forward error, and backward-error proxy $\nu(\hat{x})$, together with iteration counts and wall-clock time.

Algorithm 2 Hybrid sketched preconditioning with Polyak refinement

Require: $A, b, S, \lambda, \alpha, \beta, \tau$ $\triangleright \tau$: Cholesky condition threshold

```
1: Form  $SA$  and  $Sb$ 
2:  $G \leftarrow (SA)^\top(SA)$ ,  $h \leftarrow (SA)^\top(Sb)$ 
3: Compute a Cholesky factorization  $G = LL^\top$ 
4: if Cholesky succeeds and  $\kappa_2(G) \leq \tau$  then
5:    $z_0 \leftarrow L^{-1}h$ ,  $x^{(0)} \leftarrow (L^\top)^{-1}z_0$ 
6:    $R \leftarrow L^\top$ ,  $P \leftarrow R^{-1}$ 
7: else
8:    $\widetilde{SA} \leftarrow \begin{pmatrix} SA \\ \lambda I \end{pmatrix}$ ,  $h \leftarrow \begin{pmatrix} Sb \\ 0 \end{pmatrix}$ 
9:   Compute reduced QR:  $\widetilde{SA} = QR$ 
10:   $y_0 \leftarrow Q^\top h$ ,  $x^{(0)} \leftarrow R^{-1}y_0$ 
11:   $P \leftarrow R^{-1}$ 
12: end if
13: for  $k = 0, 1$  do
14:    $r^{(k)} \leftarrow b - Ax^{(k)}$ 
15:    $c^{(k)} \leftarrow P^\top A^\top r^{(k)}$ 
16:    $\delta y^{(k)} \leftarrow \text{SolvePolyak}(A, P, c^{(k)})$ 
17:    $x^{(k+1)} \leftarrow x^{(k)} + P\delta y^{(k)}$ 
18: end for
19: return  $x^{(2)}$ 
```

4.1 Experimental setup

For each experiment we generate synthetic least-squares instances $\min_x \|Ax - b\|_2$ as follows.

- **LS problem generation.** We construct

$$A = Q\Sigma V^\top,$$

where $Q \in \mathbb{R}^{m \times n}$ and $V \in \mathbb{R}^{n \times n}$ have orthonormal columns drawn from a random QR factorization, and $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n)$ with $\sigma_1/\sigma_n = \kappa_2(A)$ and a prescribed decay pattern for the singular values.

We draw a reference solution $x_\star$ with independent standard normal entries and set

$$b_A = Ax_\star.$$

To control the size of the orthogonal component, we draw $w \in \mathbb{R}^m$ with independent standard normal entries, project it onto the orthogonal complement of $\text{range}(A)$, and rescale it to obtain

$$b_\perp = \gamma \frac{(I - QQ^\top)w}{\|(I - QQ^\top)w\|_2},$$

where $\gamma \geq 0$ controls the relative size of the residual. The final right-hand side is

$$b = b_A + b_\perp.$$

- **Sketch and preconditioner.** For each instance we draw a sparse sign embedding $S \in \mathbb{R}^{r \times m}$ with sketch dimension $r \approx cn$, form SA and Sb , and build the preconditioner using one of the constructions described above (truncated SVD, regularized QR, Cholesky, or the hybrid strategy).

- **Solvers and stopping criteria.** All experiments are run in IEEE double precision. The inner iterative solver can be terminated either once the residual falls below a fixed tolerance or after a fixed number of iterations. In our experiment, we used a max steps of $q = 25$ and a residual tolerance of 10^{-12} . This was enough to ensure convergence, and more aggressive early termination may be possible. For our iterative solver, we use the Polyak momentum method as in the FOSSILS algorithm. We use two outer refinement steps and report the metrics for the final iterate $\hat{x}$.
- **Linear algebra routines.** All dense linear algebra operations are performed using `numpy.linalg`, which calls optimized LAPACK routines. In particular, SVD-, QR-, and Cholesky-based factorizations use `np.linalg.svd`, `np.linalg.qr`, and `np.linalg.cholesky`, respectively, while the unsketched SVD baseline is computed via `np.linalg.lstsq`.

Each reported data point is averaged over several random trials with independent draws of A , $x_\star$, and $b_\perp$.

4.2 Experiments on nearly consistent systems

For our first experiment, we test sketched solvers for nearly consistent systems. The Johnson-Lindenstrauss style bound in Eq 2 directly gives a bound on the relative residual error of the sketched solution for sketch-and-solve algorithms

$$\|b - Ax_{sk}\|_2 \leq (1 + \eta)\|b - Ax_\star\|_2, \quad (3)$$

When $\|b_\perp\|$ is on the order of machine precision, this will break down numerically. The details are discussed in Appendix A. We set the dimensions to $m = 2000, n = 50$ with a sketch size $r = 200$. We set the condition number $\kappa_2(A) = 1$ (generating an orthogonal matrix A) to minimize error amplification caused by ill-conditioning, thereby focusing purely on the impact of finite precision arithmetic on the residual. We varied the norm of the orthogonal component, $\tau := \|b_\perp\|_2$, geometrically from 10^0 down to 10^{-18} .

We solved the subsampled system $\min_x \|SAx - Sb\|_2$ using an SVD-based solver and a QR-based solver and compare the results in Figure 1.

Analysis of the noise floor. As anticipated, the empirical relative error remains essentially constant for moderate $\tau = \|b_\perp\|_2$ and then deteriorates sharply once τ approaches 10^{-13} . The breakdown does not occur at machine epsilon $u \approx 10^{-16}$, but at a larger effective noise floor where the dominant error source switches from sketch distortion to accumulated floating-point error. Appendix A shows that in finite precision any sketch-and-solve method must satisfy an inequality of the form

$$\|b_\perp\| \gtrsim \frac{cu \|b_A\|_2}{\sqrt{2\eta + \eta^2}},$$

to obtain the theoretically predicted distortion η . Here, c is a constant depending on conditioning and implementation-dependent effects. In our orthogonal test case we have $\|b_A\|_2 \approx \|x_\star\|_2 \approx 7$, and the data yielded an effective threshold

$$\|b_\perp\|_{\text{floor}} \approx \text{const} \cdot u \|x_\star\|_2 \approx 10^{-13},$$

which matches the empirical inflection point in Figure 1. Over the range of problem sizes m we tested, this observed transition level does not vary noticeably, suggesting that the constant c is relatively stable for our sketching setup.

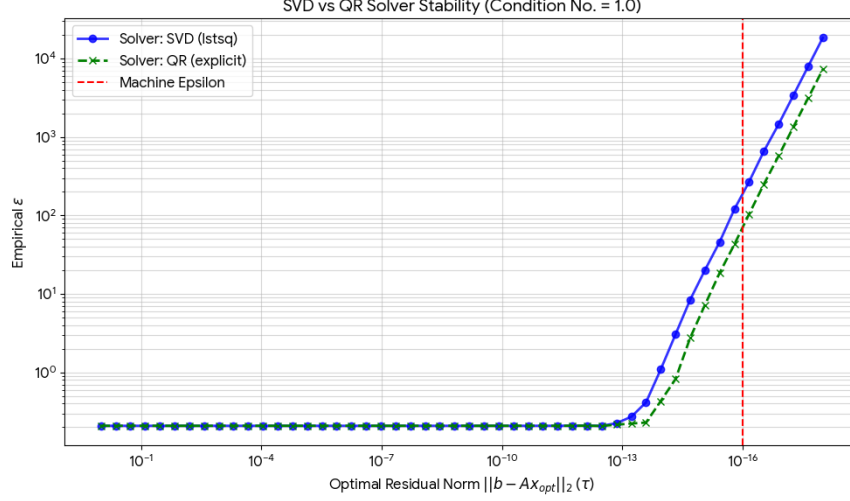


Figure 1: Comparison of SVD-based and QR-based solvers for the sketched subproblem. Both solvers exhibit a sharp breakdown in relative accuracy when the optimal residual τ drops below approximately 10^{-13} .

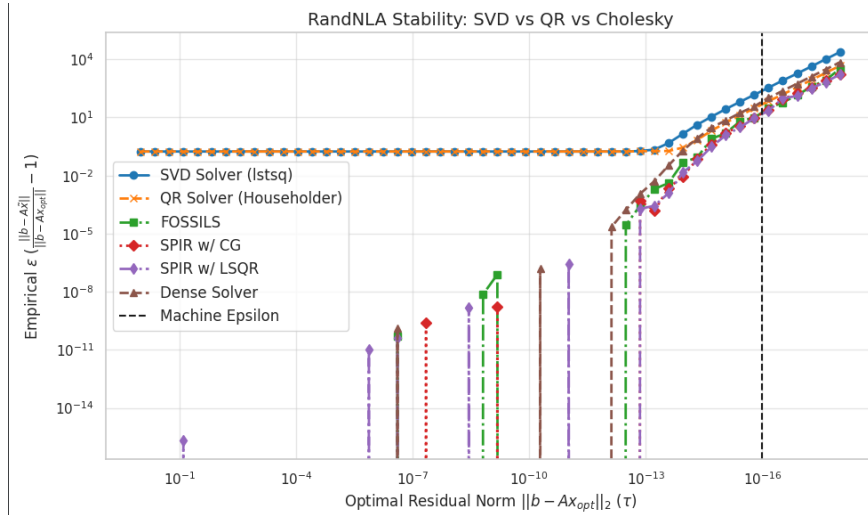


Figure 2: Comparison of all algorithms for the sketched subproblem. Although influenced by τ , algorithms with iterative refinements perform better, closer to theoretical error rate lower bound.

We also test the iterative-refinement sketching algorithms in this nearly consistent regime (Figure 2). These methods are able to drive the relative residual error down to within a constant factor of the optimal scale (for any algorithm, $\|b_\perp\|_2 \times \text{relative residual} \gtrsim u$).

SVD vs. QR Performance. We observe that the QR-based solver performs marginally better (maintaining a slightly lower error constant) than the SVD-based solver in the breakdown regime. However, the difference is negligible compared to the magnitude of the breakdown. The dominant source of numerical noise appears to be the floating-point work in forming and solving the sketched system, particularly the matrix-vector operations involving SA and Sb , which dominates the error budget regardless of the dense solver used for the subproblem.

4.3 Experiments on preconditioned iterative refinement algorithm

We test the preconditioned iterative refinement algorithms on four synthetic least-squares regimes, each specified by (m, n) and a target $\kappa_2(A)$. For every regime we run two instances: a small-residual case with $\|b_\perp\|_2 \approx 10^{-6}$ and a large-residual case with $\|b_\perp\|_2 = O(1)$. In all regimes we compare sketch + solve, truncated SVD, regularized QR, and the hybrid Cholesky/QR strategy. For each method we report the forward error, the backward-error proxy $\nu(\hat{x})$, the final residual norm, the iteration count, and the runtime. As a direct benchmark we also include an unsketched SVD solver based on `np.linalg.lstsq`, which calls highly optimized LAPACK routines. We use a sketching dimension $r = 10n$ and run a maximum of $q = 25$ steps of the Polyak inner solve.

The four regimes are:

A: $\kappa_2(A) \approx 10^4$, $m = 80,000$, $n = 2,000$; B: $\kappa_2(A) \approx 10^{12}$, $m = 80,000$, $n = 2,000$;

C: $\kappa_2(A) \approx 10^{15}$, $m = 80,000$, $n = 2,000$; D: $\kappa_2(A) \approx 10^{12}$, $m = 800,000$, $n = 200$.

Each table below corresponds to one regime. For compactness, each method appears once, with separate columns for the small- and large-residual cases.

4.3.1 Regime A: $\kappa_2(A) \approx 10^4$, $m = 80,000$, $n = 2,000$

Table 1: Regime A: small vs. large $b_\perp$.

Method	Forward err.		Backward proxy $\nu(\hat{x})$		Residual		Runtime	
	small	large	small	large	small	large	small	large
SVD (no-sketch)	1e-13	1.1e-11	7.7e-16	8e-16	3.1e-11	0	10.2	11.3
Sketch + solve	1.7e-5	1.8e1	8.3e-9	3.7e-4	5.4e-2	5.4e-2	3.6	3.8
QR	2.8e-12	4.1e-11	1.7e-15	2.7e-15	1.1e-12	0	8.8	9.2
FOSSIL (SVD)	2.8e-12	3.8e-11	1.7e-15	2.7e-15	3.1e-11	3.7e-17	9.3	11.1
Hybrid	2.8e-12	4.1e-11	1.7e-15	2.7e-15	8.5e-12	0	3.8	4.8

4.3.2 Regime B: $\kappa_2(A) \approx 10^{12}$, $m = 80,000$, $n = 2,000$

Table 2: Regime B: small vs. large $b_\perp$.

Method	Forward err.		Backward proxy $\nu(\hat{x})$		Residual		Runtime	
	small	large	small	large	small	large	small	large
SVD (no-sketch)	3.4e-1	1.2e2	1.6e-15	2.9e-16	7.6e-9	7.8e-16	10.4	10.4
Sketch + solve	1.8e2	2e8	4e-11	3.7e-11	5.1e-2	5.1e-2	3.8	3.9
QR	1.3e-1	1.2e5	3.9e-15	5.6e-16	4.1e-12	2.1e-11	9.1	9.9
FOSSIL (SVD)	1.4e-1	1.3e5	4.4e-15	1.6e-16	9.7e-12	2.6e-11	10.9	11.3
Hybrid	1.3e-1	1.2e5	4.4e-15	5.5e-16	1.9e-11	2.4e-11	10.2	11.0

4.3.3 Regime C: $\kappa_2(A) \approx 10^{15}$, $m = 80,000$, $n = 2,000$

Table 3: Regime C: small vs. large $b_\perp$.

Method	Forward err.		Backward proxy $\nu(\hat{x})$		Residual		Runtime	
	small	large	small	large	small	large	small	large
SVD (no-sketch)	5.3e-1	1.9e6	1.6e-15	2.1e-16	6.1e-9	2.2e-14	10.9	9.7
Sketch + solve	1.9e2	1.9e6	3.2e-11	3.4e-11	4e-2	4e-2	3.9	3.7
QR	1.6e3	1.5e6	1.1e-16	1.1e-16	2.3e-7	2.5e-7	10.0	11.2
FOSSIL (SVD)	3.3e3	3.4e6	9.9e-17	1.1e-16	5.5e-7	5.4e-7	10.9	11.5
Hybrid	1.6e3	1.5e6	1.1e-16	1.1e-16	2.3e-7	2.5e-7	11.1	10.6

4.3.4 Regime D: $\kappa_2(A) \approx 10^{12}$, $m = 800,000$, $n = 200$

Table 4: Regime D: small vs. large $b_\perp$.

Method	Forward err.		Backward proxy $\nu(\hat{x})$		Residual		Runtime	
	small	large	small	large	small	large	small	large
SVD (no-sketch)	4.9e-14	6.7e-12	7.6e-16	5.2e-16	3.4e-13	0	5.7	6.5
Sketch + solve	6.3e-5	5.5e1	2.5e-8	3.9e-4	5.3e-2	5.4e-2	0.0	0.1
QR	1.8e-11	6.2e-11	1.2e-14	1.1e-14	3.2e-13	1.5e-16	4.0	6.3
FOSSIL (SVD)	1.8e-11	5.9e-11	1.2e-14	1.1e-14	3e-13	0	4.0	6.2
Hybrid	1.8e-11	5.8e-11	1.2e-14	1.1e-14	1e-13	7.4e-17	3.9	6.4

4.3.5 Summary of observations

There are a few takeaways from these observations. In the experiments with $n < m < n^2$, sketching methods with iterative refinement are roughly the same speed as deterministic solvers using QR or SVD, and much faster when we can safely use Cholesky. In this regime, the dominant cost of the sketching methods is the factorization. Interestingly, while QR is slightly better than SVD, the difference is not as significant as expected. This is likely because adding regularization increases row-dimension of the sketched problem to $r + n$ rather than r .

In the experiments where $m > n^2$, the dominant cost of the FOSSILS-like methods is the iterative solver. We can see that in this case, the matrix factorization method makes minimal difference. The iterative sketching methods are roughly similar in cost to the SVD baseline in general, though this is comparing our python implementation to optimized `np.linalg.lstsq`.

In both regimes, the sketching methods obtain full backward stability but large forward error for the very inconsistent least squares problems. This indicates that the algorithm’s solutions have large components in the near-null directions of A .

References

- [1] Ethan N. Epperly, Maiké Meier, and Yuji Nakatsukasa. Fast randomized least-squares solvers can be just as accurate and stable as classical direct solvers, 2025.
- [2] Serge Gratton, Pavel Jiránek, and David Tittle-Peloquin. On the accuracy of the karlson–waldén estimate of the backward error for linear least squares problems. *SIAM Journal on Matrix Analysis and Applications*, 33:822–836, 07 2012.
- [3] Jiyan Yang, Xiangrui Meng, and Michael W. Mahoney. Implementing randomized matrix algorithms in parallel and distributed environments, 2015.

A Finite precision limits for near-consistent systems

A central result in Randomized Numerical Linear Algebra (RandNLA) is the relative-error residual bound provided by solutions to sketched LS problems. For an overdetermined least squares problem $\min_x \|Ax - b\|_2$, recall the bound give in equation

$$\|b - Ax_{sk}\|_2 \leq (1 + \eta)\|b - Ax_\star\|_2, \quad (4)$$

This is a common result used in many papers including [3] and [1].

While mathematically sound in exact arithmetic, this bound faces significant challenges in finite precision environments. We can analyze this by decomposing the residual. Suppose A is rectangular and we apply a QR decomposition such that $A = Q[R; 0]$ and $Q^T b = [b_1; b_\perp]$. The residual can be written as:

$$\|A\hat{x}_{sk} - b\|^2 = \|R\hat{x}_{sk} - b_1\|^2 + \|b_\perp\|^2.$$

Here, $\|b_\perp\|$ represents the theoretical optimal residual $\|b - Ax_\star\|$ and $\hat{x}_{sk}$ is the computed solution to the sketched problem. If we insert the theoretical bound on the computed solution, we get that

$$(1 + \eta)^2 \|b_\perp\|^2 \geq \|R\hat{x}_{sk} - b_1\|^2 + \|b_\perp\|^2.$$

However, in numerical computing, the term $\|R\hat{x} - b_1\|$ cannot vanish completely; it is lower-bounded by $u\|b_A\|$. Rearranging, we can derive a limitation on the true residual for which a distortion parameter η is obtainable:

$$\|b_\perp\| \gtrsim \frac{u\|b_A\|}{\sqrt{2\eta + \eta^2}}.$$

This inequality suggests that the linear system cannot be too consistent if we hope to obtain the desired residual error. In the Experiments section, we’ll numerically verify this and see that there is breakdown even sooner.

B Effect of regularization on preconditioner conditioning

We briefly analyze how Tikhonov regularization affects the conditioning of the factors used in our sketched preconditioners, and how this leads to a tradeoff between numerical stability and convergence behavior in iterative refinement.

B.1 Conditioning of the regularized factor

Consider the regularized normal equations associated with the stacked system

$$\min_x \left\| \begin{pmatrix} A \\ \lambda I \end{pmatrix} x - \begin{pmatrix} b \\ 0 \end{pmatrix} \right\|_2, \quad \lambda > 0,$$

whose normal matrix is $A^\top A + \lambda^2 I$. Let $\sigma_1 \geq \dots \geq \sigma_n > 0$ denote the singular values of A . If

$$A^\top A + \lambda^2 I = R^\top R$$

is a Cholesky factorization, then

$$\kappa_2(R) = \sqrt{\frac{\sigma_1^2 + \lambda^2}{\sigma_n^2 + \lambda^2}}.$$

Thus increasing λ improves the conditioning of R , making triangular solves with R^{-1} more stable in finite precision.

B.2 Conditioning of the preconditioned operator

In our algorithm, R is not used to solve the regularized problem directly, but instead defines a right preconditioner $P = R^{-1}$ for the original least-squares problem. The relevant operator for the inner iterations is therefore AR^{-1} , whose conditioning must be close to one for the FOSSILS analysis to apply.

The associated preconditioned normal operator is

$$(AR^{-1})^\top (AR^{-1}) = R^{-\top} A^\top A R^{-1}.$$

Since $R^\top R = A^\top A + \lambda^2 I$, this matrix is similar to $(A^\top A + \lambda^2 I)^{-1} A^\top A$, and hence has the same spectrum. Writing $A^\top A = V \Sigma^2 V^\top$, its eigenvalues are

$$\mu_i = \frac{\sigma_i^2}{\sigma_i^2 + \lambda^2},$$

which implies

$$\kappa_2(AR^{-1}) = \kappa_2(A) \sqrt{\frac{\sigma_n^2 + \lambda^2}{\sigma_1^2 + \lambda^2}}.$$

Thus increasing λ worsens the conditioning of the preconditioned operator.

B.3 Tradeoff and heuristic error balance

In the intermediate regime $\sigma_n \ll \lambda \ll \sigma_1$, the asymptotic scalings

$$\kappa_2(R) \approx \frac{\sigma_1}{\lambda}, \quad \kappa_2(AR^{-1}) \approx \frac{\lambda}{\sigma_n}$$

highlight a tradeoff: increasing λ worsens the conditioning of AR^{-1} , but improves the conditioning of the factor R .

In finite precision, this leads to two competing error contributions: errors introduced by repeated application of an ill-conditioned preconditioner R^{-1} , which scale roughly like $q \kappa_2(R) u$ over q inner iterations, and errors controlled by the conditioning of AR^{-1} . This balance explains why moderate regularization improves robustness, while overly aggressive regularization can degrade accuracy.